Projeto 2: Cena 3D com mesa e objetos

Luiz Felipe Neves Batista - 2311024

October 25, 2026

Sumário

1. Introdução 3
2. Aplicação básica 4
3. Fog 5
4. Plano de corte 6
5. Conclusão 7
6. Links 8
7. Uso responsável de ferramentas de IA 9

1. Introdução

Objetiva-se renderizar uma cena 3D conforme a Opção 1 do Projeto 2: uma mesa com objetos, construída por instanciação de cubos e esferas em um grafo de cena. A aplicação básica pede iluminação por fragmento, uma luz de spot, mais de um objeto com textura de padronagem de cor (decal) e um manipulador arcball associado à câmera. Como complementos, foram implementados a aplicação de fog e o plano de corte.

Foi escolhido TypeScript como a linguagem de programação, com WebGPU. Foi utilizado como base o código do professor, que implementa o grafo de cena, e o código das Tarefas 2.1 e 2.2. A cena está em `src/projeto2.ts` e é acessada pela página `projeto2.html`.

2. Aplicação básica

A cena reaproveita a mesa da Tarefa 2.2: o tampo e as quatro pernas são cubos escalados, e sobre o tampo há um tijolo amarelo (cubo), uma bola verde sobre o tijolo e uma bola vermelha (esferas). A câmera, posicionada obliquamente, recebe um manipulador arcball.

A luz de spot é a mesma da Tarefa 2.2, posicionada acima e ao lado da mesa e apontada para o centro. A iluminação é calculada por fragmento: o shader de vértice apenas transforma posição e normal para o espaço de iluminação (o espaço do mundo), e o modelo de Phong com o fator `k_spot` é calculado inteiramente no shader de fragmento. A direção da luz é calculada por pixel a partir da posição interpolada, pelo mesmo motivo discutido na Tarefa 2.2: a direção não varia linearmente ao longo de um triângulo, mas a posição sim.

Dois objetos recebem textura decal: uma folha de papel (quad) sobre a mesa e a bola verde, que recebeu uma textura de bola de futebol. Para a bola, foi criado o shader `lit_spot_textured.wgsl`, que combina a iluminação de spot por fragmento com a amostragem da textura, multiplicando a cor iluminada pela cor da textura. Um ponto que exigiu atenção foi a correspondência entre os buffers de vértice da esfera e as entradas do shader: a esfera fornece coordenada, normal, tangente e coordenada de textura, nessa ordem. Inicialmente a coordenada de textura foi lida no lugar errado e os atributos foram declarados com formato de duas componentes, o que fez a esfera aparecer como um disco achatado. A solução foi declarar a coordenada e a normal como `float32x3` e o slot da tangente com uma lista de atributos vazia, de modo que o quarto buffer alimentasse a coordenada de textura.

3. Fog

Foi implementado fog linear. Para cada fragmento, calcula-se a distância d até a câmera e o fator

$$f = \text{clamp}\left(\frac{s_{\text{fim}} - d}{s_{\text{fim}} - s_{\text{ini}}}, 0, 1\right), \tag{1}$$

que mistura a cor do objeto com a cor do fog: $c = f \cdot c_{\text{obj}} + (1 - f) \cdot c_{\text{fog}}$. A cor do fog é branca, igual à cor de fundo, de forma que objetos distantes se dissolvem no fundo.

A distância d foi obtida reaproveitando o vetor de visão. No shader de vértice, o vetor da posição até a câmera é passado ao fragmento sem normalização. Como ele é uma função afim da posição, sua interpolação é exata, e no fragmento seu comprimento fornece a distância, enquanto sua versão normalizada é usada na iluminação.

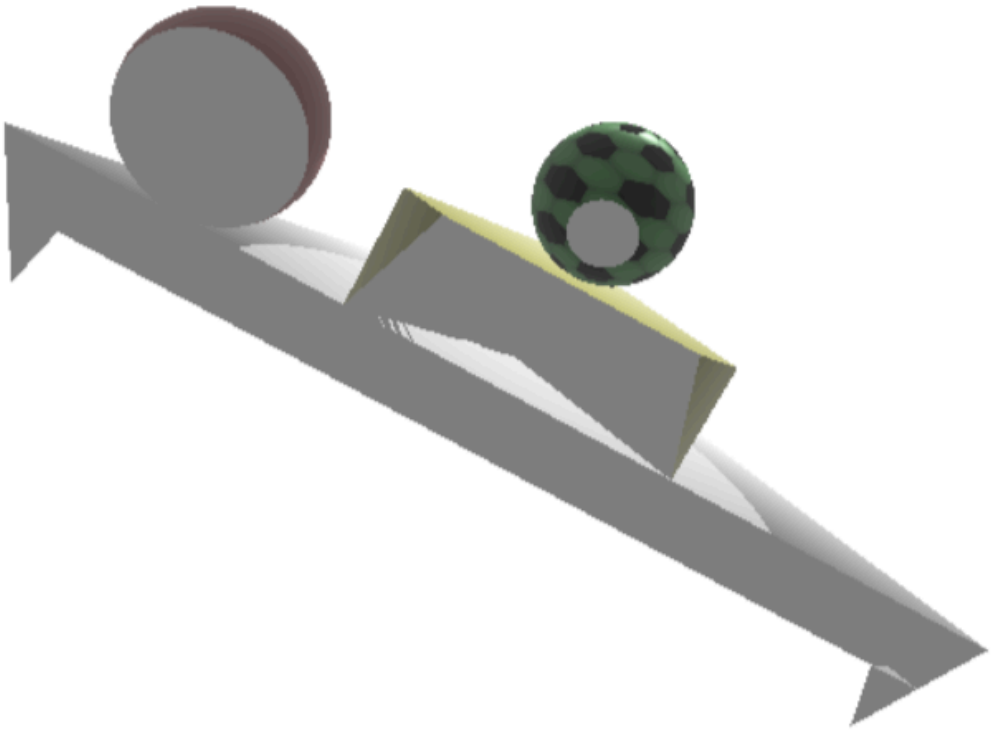
Como os shaders originais (`lit_spot.wgsl` e `textured.wgsl`) são usados pelas tarefas anteriores, foram criadas cópias exclusivas do projeto: `lit_spot_fog.wgsl` e `textured_fog.wgsl`.

4. Plano de corte

O plano de corte usa a classe `ClipPlane` do código base, que envia ao shader a equação do plano $ax + by + cz + d = 0$ como um `vec4` e a cor da seção de corte. No shader de vértice calcula-se a distância com sinal ao plano. Essa distância é interpolada e, no fragmento, os fragmentos com distância negativa são descartados.

Para que o objeto cortado pareça sólido, o culling de faces traseiras foi desativado e as faces traseiras visíveis pela abertura do corte são pintadas com a cor do corte. No quad de papel apenas se descarta o fragmento, pois ele não possui interior.

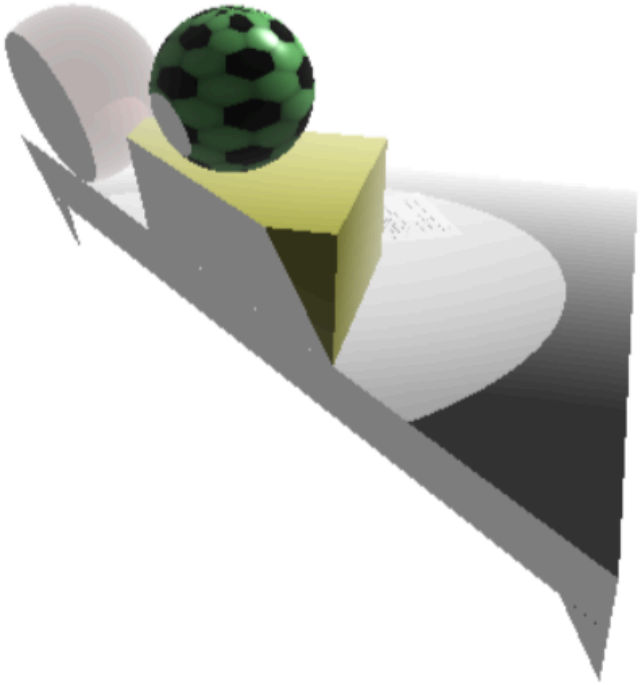
Por fim, observou-se z-fighting dentro do tijolo cortado: sua base e a face superior do tampo ocupavam exatamente o mesmo plano, $y = 0$. A seguir, um screenshot que demonstra o problema:



Antes do corte a base ficava oculta, mas com o corte ela passou a ser vista por dentro. A solução foi elevar o tijolo por um pequeno épsilon. O mesmo foi feito com as pernas da mesa em relação à face inferior do tampo.

5. Conclusão

Foi possível construir uma cena 3D complexa com aplicação de fog e o plano de corte. Segue-se um screenshot representativo da cena final:



6. Links

Link do vídeo de demonstração do funcionamento: https://drive.google.com/file/d/1O3zS3VOEyIbPNAG522fNMug11KZLDHf_/view?usp=sharing

Link do repositório: https://github.com/luizfneves404/inf1761-projeto1

7. Uso responsável de ferramentas de IA

Durante o desenvolvimento foi utilizado o assistente Claude, via Claude Code, somente para: explicação de conceitos; revisão do código de shaders e depuração; replicação mecânica de código já escrito; e redação de uma versão inicial deste relatório, revisada e ajustada cuidadosamente pelo autor. Outras partes, como a implementação da cena e da lógica do shader foram realizadas exclusivamente pelo autor.